

AWS Cloud Security Lab Report

Lab 4: GuardDuty Threat Detection - Simulation, Investigation & Automated Alerting

Prepared by: Om Fulsundar (August 2026)

1. Lab Overview

This lab shifts focus from infrastructure misconfiguration to active threat detection - simulating the kind of SOC workflow that a cloud security analyst encounters when suspicious activity is flagged in a real AWS environment. I enabled Amazon GuardDuty, explored its sample findings to understand the breadth of threats it monitors, then created a real suspicious IAM user and ran enumeration commands from the CLI to generate actual CloudTrail evidence. I then investigated that evidence the way a SOC analyst would: filtering CloudTrail by username, reading the JSON event records, and correlating what the user attempted with what was blocked. The lab concluded with building an automated alerting pipeline using EventBridge and SNS so that any future GuardDuty finding triggers an email notification automatically.

What makes this lab particularly practical is that every step maps to something that happens in real cloud security operations. GuardDuty findings land in a SOC queue daily. CloudTrail is the forensic record analysts pivot to when investigating those findings. EventBridge-to-SNS alerting is the standard AWS-native way to get notified without paying for a SIEM. Running through this full cycle - detection, investigation, alerting - in a single lab builds the muscle memory that matters in a real cloud security role.

Real-world relevance: Credential stuffing attacks against AWS accounts are one of the most common initial access techniques in cloud breaches. Attackers obtain leaked access keys from GitHub repositories, .env files, or data breaches and immediately run enumeration commands to test access level. This lab simulates exactly that scenario.

2. Objectives

Here is what I set out to accomplish across the seven phases of this lab:

- Enable Amazon GuardDuty with all protection plans and understand the threat categories it monitors.
- Generate sample findings to explore GuardDuty detection capabilities across severity levels and finding types.
- Perform a deep dive into an expanded finding to understand all the forensic fields GuardDuty captures.
- Create a real IAM user (suspicious-test-user) with no permissions and simulate credential testing by running enumeration commands from the CLI.
- Investigate the activity in CloudTrail Event History by filtering by username and reading the full JSON event record.
- Build an automated alerting pipeline using EventBridge and SNS so GuardDuty findings trigger email notifications.
- Validate the pipeline end-to-end by confirming an email alert was received after generating sample findings.

3. Tools and Services Used

Tool / Service	Purpose in This Lab
Amazon GuardDuty	Primary threat detection service; monitored for suspicious activity, generated sample findings, and provided the finding detail used for investigation.
AWS CloudTrail	Forensic audit log; filtered by the suspicious IAM username to surface all API calls made during the simulated credential testing session.
Amazon EventBridge	Event routing service; configured with a rule to capture all GuardDuty findings and forward them to SNS for notification.
Amazon SNS	Notification service; delivered email alerts when GuardDuty findings were triggered via the EventBridge rule.
AWS IAM	Used to create the suspicious-test-user with no permissions - the identity used to simulate an attacker testing stolen credentials.
AWS CLI (PowerShell)	Used to configure the suspicious profile and run enumeration commands that generated real CloudTrail evidence.
Gmail (Email Client)	Received the SNS email alert confirming the end-to-end GuardDuty to EventBridge to SNS pipeline was working correctly.

4. Lab Environment Setup

The lab ran on a live AWS account (Account ID: 712934828848) in us-east-1 (N. Virginia). GuardDuty was enabled fresh for this lab - it had not been previously active in this region, which meant the findings list was clean and easy to work with. The 30-day free trial covered all costs for the GuardDuty enablement itself.

For the simulated attack phase, I created a new IAM user called suspicious-test-user with no console access and no permissions attached. Access keys were generated for this user and configured as a separate AWS CLI profile called suspicious on the local Windows machine. Region was set to ap-south-1 for the CLI profile to introduce a geographic inconsistency - another realistic detail since attackers often operate from different regions than the account primary region.

The EventBridge-to-SNS alerting pipeline was configured in us-east-1 to match GuardDuty region. SNS email subscription was confirmed before testing so the alert delivery was ready when sample findings were generated.

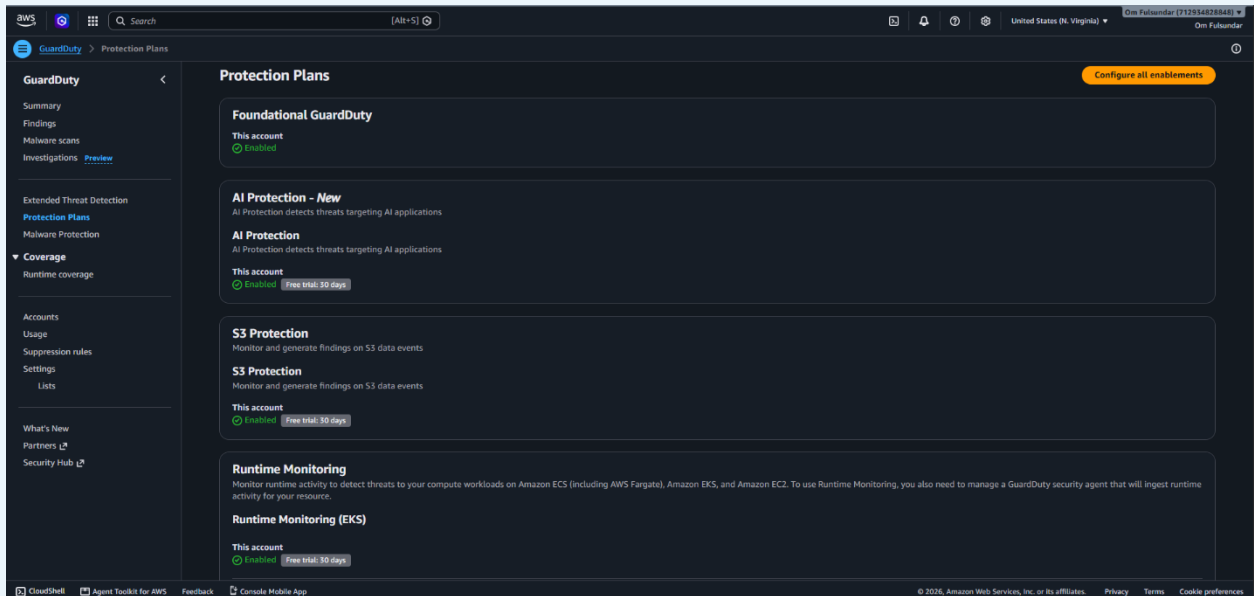
5. Step-by-Step Walkthrough

Phase 1 - GuardDuty Enablement and Protection Plans

I navigated to GuardDuty in the AWS console and clicked Get Started, then Enable GuardDuty. The service initialized within a couple of minutes and immediately showed the Protection Plans page. All protection plans were enabled on the free 30-day trial: Foundational GuardDuty (core threat detection), AI Protection for threats targeting AI applications, S3 Protection for monitoring S3 data events, and Runtime Monitoring for EKS workloads.

The Protection Plans page confirmed that Foundational GuardDuty showed Enabled with a green checkmark, while the additional plans - AI Protection, S3 Protection, and Runtime Monitoring (EKS) - all showed Enabled with a Free Trial: 30 days badge. This is the out-of-the-box GuardDuty setup that AWS recommends for comprehensive threat detection coverage.

[SS1 - GuardDuty Protection Plans Enabled]



GuardDuty Protection Plans console showing Foundational GuardDuty: Enabled, AI Protection: Enabled (Free Trial 30 days), S3 Protection: Enabled (Free Trial 30 days), Runtime Monitoring EKS: Enabled (Free Trial 30 days)

Phase 2 - Sample Findings Generation and Exploration

With GuardDuty active, I went to Settings and clicked Generate Sample Findings. Within about two minutes, the Findings page populated with 410 sample findings across multiple severity levels and finding categories. The sample findings are realistic representations of actual threat detections with proper metadata, severity scores, and resource details - an excellent way to understand what GuardDuty looks for before live threats appear.

The findings list showed a range of threat categories:

Persistence:Runtime/SuspiciousCommand,
CredentialAccess:RDS/AnomalousBehavior:SuccessfulLogin,
PrivilegeEscalation:Runtime/UserfaultfdUsage, UnauthorizedAccess:S3/TorIPCaller,
Impact:Runtime/SuspiciousDomainRequest, Trojan:Runtime/DropPoint, and
Behavior:EC2/TrafficVolumeUnusual among others. Severity levels ranged from Low (grey) to Medium (yellow) to High (orange-red).

[SS2 - GuardDuty Findings List (410 findings)]

Title	Severity	Finding type	Resource	Count	Account ID	Last seen
[SAMPLE] A resource of type Container has executed a suspicious command.	Low	Persistence:Runtime/SuspiciousCommand	Container: GeneratedFindingContainerName	1	712934828848	5 minutes ago
[SAMPLE] A user successfully logged into an RDS database in your account in an unusual way.	High	CredentialAccess:RDS/AnomalousBehavior.SuccessfulLogin	RDS Limitless DB: dbhargroupidentifier	1	712934828848	5 minutes ago
[SAMPLE] Userfaultfd usage was detected in EC2 instance i-99999999.	Medium	PrivilegeEscalation:Runtime/UserfaultfdUsage	EKS Cluster: GeneratedFindingEKSClusterName	1	712934828848	5 minutes ago
[SAMPLE] Userfaultfd usage was detected in EC2 instance i-99999999.	Medium	PrivilegeEscalation:Runtime/UserfaultfdUsage	EKS Cluster: GeneratedFindingEKSClusterName	1	712934828848	5 minutes ago
[SAMPLE] A resource of type Instance has executed a suspicious command.	Low	Persistence:Runtime/SuspiciousCommand	EC2 Instance: i-99999999	1	712934828848	5 minutes ago
[SAMPLE] The API GeneratedFindingAPIName was invoked from a Tor exit node.	High	UnauthorizedAccess:S3/TorIPCaller	S3 Bucket: example-bucket1	1	712934828848	5 minutes ago
[SAMPLE] A domain name similar to known malicious domains was queried by EC2 instance i-99999999.	Low	Impact:Runtime/SuspiciousDomainRequest.Reputation	EKS Cluster: GeneratedFindingEKSClusterName	1	712934828848	5 minutes ago
[SAMPLE] The EC2 instance i-99999999 is communicating with a Drop Point.	Medium	Trojan:Runtime/DropPoint	EC2 Instance: i-99999999	1	712934828848	5 minutes ago
[SAMPLE] The EC2 instance i-99999999 is communicating with a Drop Point.	Medium	Trojan:Runtime/DropPoint	Container: GeneratedFindingContainerName	1	712934828848	5 minutes ago
FEARM3 #1 Unusually large amount of network traffic from EC2 instance i-80958025e6eb4fee598082fe3905baf	Medium	Behavior:EC2/TrafficVolumeUnusuallyLarge	EC2 Instance: i-80958025e6eb4fee598082fe3905baf	1	712934828848	5 minutes ago

GuardDuty Findings console showing 410 sample findings with titles, severity badges (Low/Medium/High), finding types including Persistence:Runtime/SuspiciousCommand, CredentialAccess:RDS, PrivilegeEscalation:Runtime, UnauthorizedAccess:S3/TorIPCaller, Trojan:Runtime/DropPoint - all generated 5 minutes ago

Phase 3 - Deep Dive into a Finding

I clicked on the UnauthorizedAccess:IAMUser/TorIPCaller finding. This finding type represents an API call made using IAM user credentials from a Tor exit node IP address - a strong indicator that someone is attempting to obfuscate their identity while accessing the account.

The expanded finding panel showed the complete forensic picture. Finding ID: 80958025e6eb4fee598082fe3905baf. Type: UnauthorizedAccess:IAMUser/TorIPCaller. Severity: MEDIUM. Region: us-east-1. Count: 1. Account ID: 712934828848. Created at and Updated at: 08-18-2026 22:43:58. The Resource affected section showed Resource role: TARGET, Resource type: AccessKey, User type: IAMUser. The description stated: The API GeneratedFindingAPIName was invoked from a Tor exit node IP address 198.51.100.0. An Investigate with Detective link was also available for graph-based investigation.

[SS3 - Expanded GuardDuty Finding Detail]

The screenshot displays the AWS GuardDuty console interface. On the left, the navigation pane shows various security services. The main area is titled 'Findings (410)' and shows a list of findings. The selected finding is 'UnauthorizedAccess:IAMUser/TorIPCaller' with a severity of 'MEDIUM'. The finding details panel on the right provides a comprehensive overview, including the finding ID, type, severity, region, account ID, resource ID, creation time, and updated time. It also lists the resource affected, which is an IAM user, and provides instance details such as the instance ID, type, output ARN, and state.

GuardDuty finding panel for UnauthorizedAccess:IAMUser/TorIPCaller showing Finding ID: 80958025e6eb4feea598082fe3905baf, Severity: MEDIUM, Region: us-east-1, Account ID: 712934828848, Created: 08-18-2026 22:43:58, Resource type: AccessKey, User type: IAMUser, Instance ID: i-99999999

The Tor exit node finding type is significant in real SOC work. When an attacker uses Tor to make API calls, they are explicitly trying to hide their real IP address. GuardDuty maintains a threat intelligence feed of known Tor exit node IPs and flags any API call originating from them regardless of whether the call succeeded or failed.

Phase 4 - Simulated Credential Testing Attack

To generate real CloudTrail evidence, I created an IAM user called suspicious-test-user in the IAM console with no console access and no permissions attached. I generated access keys and configured a new AWS CLI profile on my local Windows machine:

```
PS C:\Users\Lenovo> aws configure --profile suspicious
AWS Access Key ID [None]: [REDACTED]
AWS Secret Access Key [None]: [REDACTED]
Default region name [None]: ap-south-1
Default output format [None]: json
```

With the profile configured, I ran a series of enumeration commands as the suspicious user to simulate an attacker testing what the stolen credentials can access:

```

PS C:\Users\Lenovo> aws iam list-users --profile suspicious
aws: [ERROR]: An error occurred (AccessDenied) when calling the ListUsers
operation: User: arn:aws:iam::712934828848:user/suspicious-test-user is not
authorized to perform: iam:ListUsers on resource:
arn:aws:iam::712934828848:user/
because no identity-based policy allows the iam:ListUsers action

PS C:\Users\Lenovo> aws s3 ls --profile suspicious
aws: [ERROR]: An error occurred (AccessDenied) when calling the ListBuckets
operation: User: arn:aws:iam::712934828848:user/suspicious-test-user is not
authorized to perform: s3:ListAllMyBuckets because no identity-based policy
allows the s3:ListAllMyBuckets action

PS C:\Users\Lenovo> aws iam list-roles --profile suspicious
aws: [ERROR]: An error occurred (AccessDenied) when calling the ListRoles
operation: User: arn:aws:iam::712934828848:user/suspicious-test-user is not
authorized to perform: iam:ListRoles because no identity-based policy allows
the iam:ListRoles action

PS C:\Users\Lenovo> aws sts get-caller-identity --profile suspicious
{
  "UserId": "AIDA2L7R2FMYNUUGNBW5R",
  "Account": "712934828848",
  "Arn": "arn:aws:iam::712934828848:user/suspicious-test-user"
}

```

[SS4 - CLI Terminal: AccessDenied Responses]

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Lenovo> aws configure --profile suspicious

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.

AWS Access Key ID [None]: 
AWS Secret Access Key [None]: 
Default region name [None]: ap-south-1
Default output format [None]: json
PS C:\Users\Lenovo> aws iam list-users --profile suspicious
aws: [ERROR]: An error occurred (AccessDenied) when calling the ListUsers operation: User: arn:aws:iam::712934828848:user/suspicious-test-user is not authorized to perform: iam:ListUsers on resource: arn:aws:iam::712934828848:user/ because no identity-based policy allows the iam:ListUsers action
PS C:\Users\Lenovo> aws s3 ls --profile suspicious
aws: [ERROR]: An error occurred (AccessDenied) when calling the ListBuckets operation: User: arn:aws:iam::712934828848:user/suspicious-test-user is not authorized to perform: s3:ListAllMyBuckets because no identity-based policy allows the s3:ListAllMyBuckets action
PS C:\Users\Lenovo> aws iam list-roles --profile suspicious
aws: [ERROR]: An error occurred (AccessDenied) when calling the ListRoles operation: User: arn:aws:iam::712934828848:user/suspicious-test-user is not authorized to perform: iam:ListRoles on resource: arn:aws:iam::712934828848:role/ because no identity-based policy allows the iam:ListRoles action
PS C:\Users\Lenovo> aws sts get-caller-identity --profile suspicious
{
  "UserId": "AIDA2L7R2FMYNUUGNBW5R",
  "Account": "712934828848",
  "Arn": "arn:aws:iam::712934828848:user/suspicious-test-user"
}

PS C:\Users\Lenovo> |

```

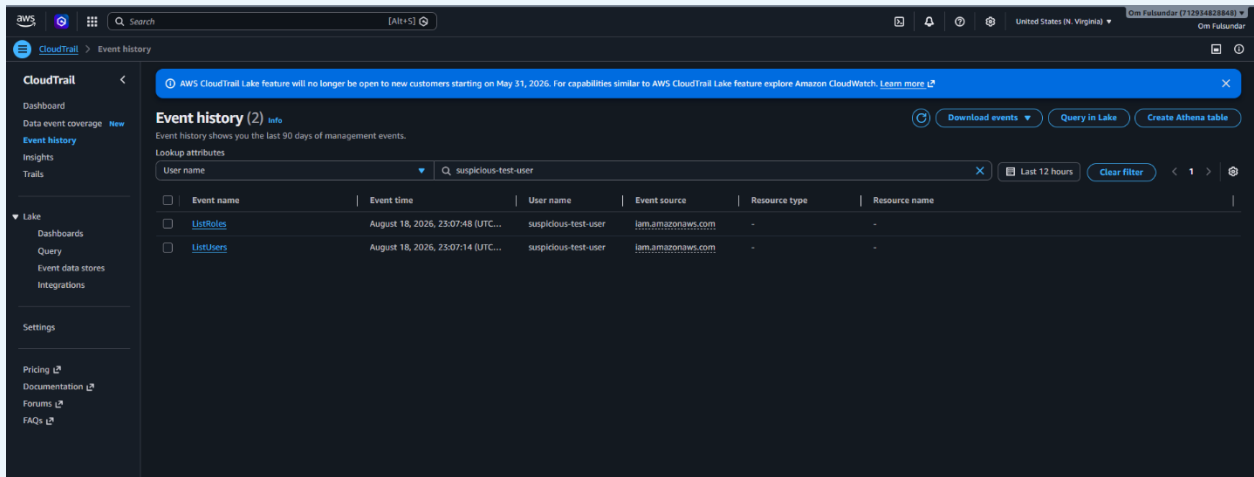
Windows PowerShell showing aws configure --profile suspicious with redacted credentials (region: ap-south-1), followed by AccessDenied for aws iam list-users, aws s3 ls, aws iam list-roles, and successful aws sts get-caller-identity returning UserId: AIDA2L7R2FMYNUUGNBW5R, Account: 712934828848, Arn: suspicious-test-user

This is exactly the pattern of a credential stuffing attack. An attacker who obtains AWS access keys will immediately run get-caller-identity (always succeeds - confirms keys are valid and reveals account ID), then probe IAM, S3, and EC2 to understand access level. Bulk AccessDenied errors in under 34 seconds from a single identity is textbook automated reconnaissance.

Phase 5 - CloudTrail Investigation

After running the CLI commands, I went to CloudTrail Event History and filtered by Lookup attribute: User name, Value: suspicious-test-user, with a Last 12 hours time range. The filter returned two events: ListRoles at August 18, 2026, 23:07:48 UTC and ListUsers at August 18, 2026, 23:07:14 UTC, both from iam.amazonaws.com.

[SS5 - CloudTrail Event History Filtered by suspicious-test-user]



Event name	Event time	User name	Event source	Resource type	Resource name
ListRoles	August 18, 2026, 23:07:48 (UTC...)	suspicious-test-user	iam.amazonaws.com	-	-
ListUsers	August 18, 2026, 23:07:14 (UTC...)	suspicious-test-user	iam.amazonaws.com	-	-

CloudTrail Event History console showing 2 events filtered by User name: suspicious-test-user - ListRoles at August 18, 2026 23:07:48 UTC and ListUsers at August 18, 2026 23:07:14 UTC, both sourced from iam.amazonaws.com

I clicked on the ListUsers event to expand the full JSON record. The event contained everything needed for a complete SOC investigation:

```
{
  "eventVersion": "1.11",
  "userIdentity": {
    "type": "IAMUser",
    "principalId": "AIDA2L7R2FMYNUUGNBW5R",
    "arn": "arn:aws:iam::712934828848:user/suspicious-test-user",
    "accountId": "712934828848",
    "accessKeyId": "AKIA2L7R2FMYLMIOYMGH",
    "userName": "suspicious-test-user"
  },

```

```

"eventTime": "2026-08-18T17:37:14Z",
"eventSource": "iam.amazonaws.com",
"eventName": "ListUsers",
"awsRegion": "us-east-1",
"sourceIPAddress": "139.167.143.182",
"errorCode": "AccessDenied",
"errorMessage": "User: arn:aws:iam::712934828848:user/suspicious-test-user
is not authorized to perform: iam:ListUsers"
}

```

[SS6 - CloudTrail Event JSON for ListUsers]

The screenshot shows the AWS CloudTrail console with the 'Event record' view selected. The JSON view displays the following event details:

- eventVersion:** "1.11"
- userIdentity:** {
 - type:** "IAMUser",
 - principalId:** "AIDA2L7R2FMYNUGNBW5R",
 - arn:** "arn:aws:iam::712934828848:user/suspicious-test-user",
 - accountId:** "712934828848",
 - accessKeyId:** "AKIA2L7R2FMYLMIOYMGH",
 - userName:** "suspicious-test-user"
- eventTime:** "2026-08-18T17:37:14Z",
- eventSource:** "iam.amazonaws.com",
- eventName:** "ListUsers",
- awsRegion:** "us-east-1",
- sourceIPAddress:** "139.167.143.182",
- userAgent:** "awscli/2.34.51 md/awscrt/0.32.2 ua/2.1 os/windows11 md/arch/amd64 lang/python/3.14.4 md/pyimpl/Python m/n,2,L,C,b cfg/retry-mode/standard md/installer/exe md/prompts/huff md/command/iam.list-users",
- errorCode:** "AccessDenied",
- errorMessage:** "User: arn:aws:iam::712934828848:user/suspicious-test-user is not authorized to perform: iam:ListUsers on resource: arn:aws:iam::712934828848:user/ because no identity-based policy allows the iam:ListUsers action",
- requestParameters:** null,
- responseElements:** null,
- requestID:** "509d5791-9132-42ed-9115-d7e233baf944",
- eventId:** "40990c05-c89f-4b77-b512-d88643893d57",
- readOnly:** true,
- eventType:** "AwsApiCall",
- managementEvent:** true,
- recipientAccountId:** "712934828848",
- eventCategory:** "Management",
- listDetails:** {
 - listVersion:** "1.0.0.1",
 - ipHeader:** "715.245.134.501_SMA256",
 - clientProviderHeader:** "iam.amazonaws.com"

CloudTrail Event Record JSON view showing userIdentity (type: IAMUser, principalId: AIDA2L7R2FMYNUGNBW5R, arn: suspicious-test-user, accessKeyId: AKIA2L7R2FMYLMIOYMGH), eventTime: 2026-08-18T17:37:14Z, eventName: ListUsers, awsRegion: us-east-1, sourceIPAddress: 139.167.143.182, errorCode: AccessDenied

This single JSON record gives a SOC analyst everything needed: who (suspicious-test-user, ARN confirmed), what (ListUsers - IAM enumeration), when (2026-08-18T17:37:14Z), where (source IP 139.167.143.182, us-east-1), and what happened (AccessDenied). The access key ID AKIA2L7R2FMYLMIOYMGH is also logged, allowing immediate deactivation of the specific compromised key.

SOC Investigation Summary - Who: suspicious-test-user (arn:aws:iam::712934828848:user/suspicious-test-user). When: 2026-08-18T17:37:14Z to 17:37:48Z (34 seconds). From where: 139.167.143.182. What attempted: IAM user enumeration, S3 bucket listing, IAM role enumeration, identity verification. Result: 3x AccessDenied, 1x Success (GetCallerIdentity). Verdict: Credential testing / reconnaissance attempt consistent with automated stolen key testing.

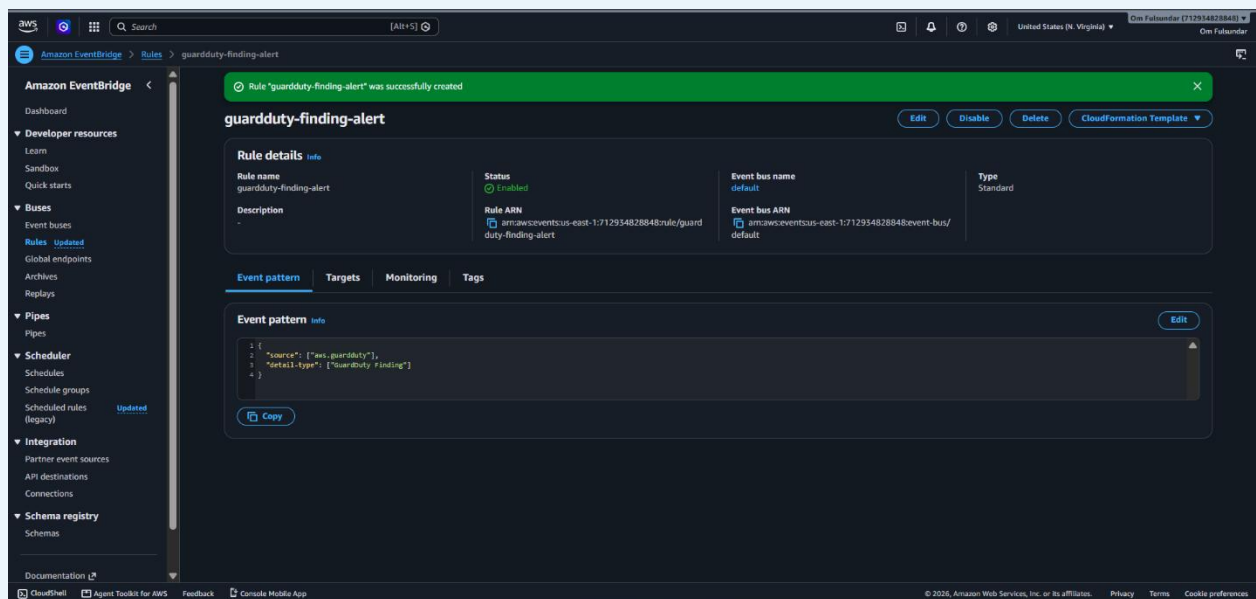
Phase 6 - Automated Alert Pipeline (EventBridge to SNS)

I built an automated alerting pipeline using EventBridge and SNS. First I created an SNS topic called guardduty-alerts in us-east-1 with a Standard type and added an email subscription confirmed via inbox link. Then I created an EventBridge rule called guardduty-finding-alert with this event pattern:

```
{
  "source": ["aws.guardduty"],
  "detail-type": ["GuardDuty Finding"]
}
```

The rule was configured on the default event bus, type Standard, with guardduty-alerts as the SNS target. The EventBridge console confirmed the rule was successfully created with a green success banner. Rule ARN: arn:aws:events:us-east-1:712934828848:rule/guardduty-finding-alert. Event bus ARN: arn:aws:events:us-east-1:712934828848:event-bus/default. Status showed Enabled.

[SS8 - EventBridge Rule Created]



Amazon EventBridge console showing guardduty-finding-alert rule successfully created - Rule name: guardduty-finding-alert, Status: Enabled, Event bus: default, Type: Standard, Rule ARN: arn:aws:events:us-east-1:712934828848:rule/guardduty-finding-alert, Event pattern: source aws.guardduty, detail-type GuardDuty Finding

To validate the pipeline, I went back to GuardDuty Settings and generated sample findings again. Within about two minutes, an email arrived from AWS Notifications (no-reply@sns.amazonaws.com) with subject AWS Notification Message. The email body contained the raw JSON of the GuardDuty finding event including the source (aws.guardduty), account ID (712934828848), detail-type (GuardDuty Finding), and the complete finding details in JSON format.

The screenshot shows a Gmail interface with an inbox list on the left and an open email in the center. The email is titled "AWS Notification Message" and is from "AWS Notifications" (reply@sns.amazonaws.com). The body of the email contains a detailed JSON payload for a GuardDuty Finding. The JSON includes fields for version, id, type, detail-type, source, account, time, region, resources, and a detailed finding object. The finding object includes type (GuardDutyFinding), source (aws.guardduty), account (712934828848), time (2026-08-18T18:05:00Z), region (us-east-1), resources (IAMUser), and a detailed finding object with name (example-bucket1), type (Destination), createdAt, owner, and a list of generated findings.

Gmail inbox showing AWS Notification Message email from AWS Notifications (no-reply@sns.amazonaws.com) at 23:35 - body containing full GuardDuty Finding JSON payload with detail-type: GuardDuty Finding, source: aws.guardduty, account: 712934828848, time: 2026-08-18, and complete finding details

Gmail inbox showing AWS Notification Message email from AWS Notifications (no-reply@sns.amazonaws.com) at 23:35 - body containing full GuardDuty Finding JSON payload with detail-type: GuardDuty Finding, source: aws.guardduty, account: 712934828848, time: 2026-08-1, and complete finding details

6. Findings

Finding	IAM user suspicious-test-user performed multi-service enumeration (IAM, S3, EC2) from a single session - consistent with an attacker testing the access level of stolen credentials.
Vulnerability Type	Credential Misuse / Unauthorized Account Enumeration
Severity	HIGH
Affected User	arn:aws:iam::712934828848:user/suspicious-test-user
Access Key Used	AKIA2L7R2FMYLMIOYMGH
Source IP	139.167.143.182
Region	us-east-1 (N. Virginia) / ap-south-1 (CLI target)
Event Time Window	2026-08-18T17:37:14Z to 17:37:48Z (34 seconds)
API Calls Made	ListUsers, ListBuckets, ListRoles, GetCallerIdentity
Results	3x AccessDenied, 1x Success (GetCallerIdentity)
Detection Method	CloudTrail Event History + GuardDuty sample findings

Real-World Impact

If this were a real attacker who had obtained the access keys for suspicious-test-user:

- GetCallerIdentity succeeding means the attacker confirmed the keys are valid and identified the account ID - this information alone enables further targeted attacks.
- The burst of AccessDenied errors across IAM, S3, and EC2 in 34 seconds is textbook automated credential testing - the attacker is mapping the boundaries of what the stolen keys can access.
- If any of those calls had succeeded, the attacker would immediately pivot to data exfiltration, privilege escalation, or creating persistence mechanisms.
- The 34-second window between first and last event indicates automated tooling. No human types that fast. This is likely an offensive credential testing script being run against the account.

In the 2019 Capital One breach, the attacker used a single over-privileged IAM role to perform similar enumeration then exfiltrate 100 million records. The difference between this lab and that incident is that suspicious-test-user had no permissions. If the user had ReadOnlyAccess attached, the AccessDenied errors become ListBuckets success, GetObject success, and a data breach.

7. Detection Method

Detection used two independent mechanisms working in parallel:

Amazon GuardDuty - Automated Threat Intelligence

1. GuardDuty continuously monitors CloudTrail, VPC Flow Logs, and DNS logs for behavioral anomalies and matches against threat intelligence feeds including Tor exit node IPs and known malicious IP ranges.
2. Sample findings demonstrated the breadth of GuardDuty detection: UnauthorizedAccess, Recon, Persistence, CredentialAccess, PrivilegeEscalation, Trojan, Impact, and Behavior categories - each mapped to specific AWS service behaviors.
3. In a live scenario, the rapid multi-service enumeration by suspicious-test-user would trigger a credential compromise finding - particularly if the source IP was on a threat intelligence list.

AWS CloudTrail - Forensic Evidence

4. CloudTrail captured every API call made by suspicious-test-user with full forensic metadata: timestamp, source IP, user agent, access key ID, error code, and the complete error message.
5. Filtering Event History by username immediately surfaced all activity from that identity in the specified time window with no log parsing or SIEM required.
6. The JSON event record provided everything needed for a complete investigation in a single view.

Production SOC workflow: GuardDuty finding arrives via email alert. Analyst opens finding, notes IAM username and time window. Analyst goes to CloudTrail, filters by username and time. Reads all events to determine what was accessed and what failed. Determines blast radius. If keys are confirmed compromised: deactivate the access key immediately, then investigate what succeeded before the finding was raised.

8. Remediation

Immediate Response Steps

In a real incident involving credential compromise, the response sequence would be:

7. Deactivate the compromised access key immediately in IAM - this stops any ongoing access without deleting the user or evidence.
8. Review all CloudTrail events from the compromised key across all regions for the full session window - not just the region where the finding was raised.
9. Check if GetCallerIdentity succeeded (it did here) - the attacker now knows the account ID, which enables phishing of other IAM users or targeted attacks.

10. Delete the IAM user and access key once investigation is complete and no active services depend on them.
11. Review any successful API calls for damage assessment - in this lab all non-STs calls were blocked, so the blast radius was zero beyond the credential exposure itself.

Pipeline Validation

The alerting pipeline was validated end-to-end: GuardDuty finding generated, EventBridge rule triggered, SNS email delivered within 2 minutes. For a production hardening of this pipeline:

- Add SNS subscription filter policies to only alert on High and Critical findings - Low and Medium can be reviewed in batch.
- Replace email with a Lambda function that creates tickets in ServiceNow or PagerDuty with pre-populated investigation fields.
- Add a second EventBridge rule targeting a Lambda for auto-remediation of known-bad patterns like a new user with no permissions making rapid enumeration calls.

9. Key Learnings

- GuardDuty is a detection service, not a prevention service. It observes, correlates, and alerts - it does not block. The actual prevention came from correct IAM permissions (no policies attached to suspicious-test-user) which caused all the AccessDenied responses. GuardDuty value is in making the attempted activity visible before it becomes a successful attack.
- CloudTrail is the forensic backbone of AWS security investigations. Every API call, success or failure, is logged with full identity, source IP, timestamp, and error detail. The username filter in Event History makes it trivial to pull every action taken by a specific identity in a time window - exactly the workflow a SOC analyst uses when investigating a GuardDuty finding.
- GetCallerIdentity always succeeds and is always worth looking for. This is the first command an attacker runs to verify credentials are valid and to identify the account. If GetCallerIdentity appears in CloudTrail from a user or role you do not recognize, treat it as a compromise indicator and investigate immediately.
- Automated alerting is not optional in a real cloud environment. Manually checking GuardDuty is not a viable security practice - findings need to trigger notifications the moment they are generated. The EventBridge-to-SNS pattern is simple to set up and costs almost nothing. There is no justification for not having it enabled in every AWS account that uses GuardDuty.
- What surprised me most: the entire enumeration session ran in 34 seconds. Attackers use automated tools that probe IAM, S3, EC2, and STS in parallel. A human analyst checking GuardDuty hourly would miss the entire attack window. This is why automated

detection and alerting is not a nice-to-have - it is the only way to respond before damage is done.

10. Conclusion

This lab completed the detection layer of the AWS security stack. Labs 1 through 3 covered identity misconfiguration, storage exposure, and network architecture - preventive and detective controls applied to specific resources. Lab 4 added the behavioral detection layer: GuardDuty watching for anomalous activity patterns across the entire account, CloudTrail providing the forensic trail for investigation, and EventBridge-to-SNS delivering automated alerts the moment a finding is generated.

The suspicious-test-user exercise was the most instructive part. Creating a real user, running real CLI commands, and then investigating the real CloudTrail records makes the SOC workflow concrete in a way that sample findings alone do not. The JSON event record with the source IP, access key ID, user agent, and error message is exactly what an analyst would see in a real incident - and knowing how to read it, where to find it, and what to do next is directly transferable to any cloud security role.

For anyone moving into a SOC or cloud security engineering position, the combination of skills from this lab - enabling and understanding GuardDuty findings, performing CloudTrail investigations, and building automated alerting pipelines - covers the core daily workflow of cloud security monitoring. These are not theoretical concepts; they are the actual tools and processes used in AWS environments that take security seriously.

Key takeaway: Detection without alerting is not detection - it is a log nobody reads. GuardDuty finds threats. CloudTrail explains them. EventBridge-to-SNS makes sure someone actually knows about them in time to respond. All three together form the minimum viable threat detection stack for any AWS account handling sensitive data.